1 · Runtime architecture and data contracts



Fact source, evidence selection and prose generation are separate stages.

Request trace — GET /api/games/{id}/recap → local/Redis cache → ESPN game+summary → normalized facts → key-play selection → writer or template → labeled response. Keep fetching, classification, ranking, writing and persistence distinct. The live path is not automatically the historical PostgreSQL ingestion path elsewhere in the repo.

espn_public.py — _event_to_game requires competition, home/away participants and event ID; outputs id, sport, status, date, teams and nullable scores. _score_value returns None for missing/invalid values. A missing score is unknown, not zero; zero is a valid observed score. _get_json uses URL+query key, timeout=12s and raises for HTTP failures.

Adapter boundary — Normalize provider-specific nesting once so downstream code consumes stable fields. Fixtures should cover missing participants, invalid IDs, postponed/live/final games and corrected scores. Preserve source timestamp and event identity; a fresh HTTP response is not evidence that upstream facts are final.

2 · Cache semantics and concurrent misses

Layer	Behavior / boundary
Source JSON	Process-local URL+query cache; lock around access
Recap memo	recap:v2:{game_id}; six-hour TTL; size clear
Redis	Shared when configured; separate from local memo
Eligibility	Both scores, not live, and LLM success or no LLM configured

Current behavior — get_recap already avoids caching a data fallback when an LLM is configured. That prevents a transient provider failure from pinning the inferior response. Memo and source locks protect get/set, but release before fetching/generation: simultaneous misses can still duplicate expensive work.

```
hit? return hit
miss -> fetch -> build -> eligibility check -> cache
# No per-key single-flight covers this entire path.
```

What to improve — Key lacks source revision, prompt and model revision. Corrected final data can reuse stale prose. Add a fact fingerprint + schema/prompt version; coalesce work per key; use explicit stale/refresh policy. Cache shared game facts separately from personalized results. Compare hit/miss latency and cost, not one blended average.

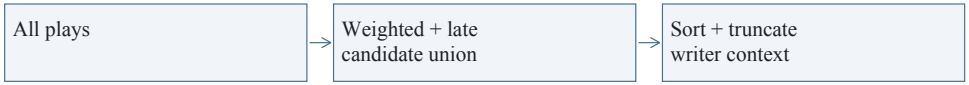
3 · Play analysis: rules behind the agent names

Stable interface — PlayContext holds sport, description, period/clock, current/previous scores and optional field position. analyze(ctx) returns AgentRead with headline/detail/metric. Scout, Stat and other modules are deterministic rules, not trained CV models. The contract allows later replacements with independently tested learned implementations.

$$\text{Kick proxy } p = 1/[1+\exp((\text{distance}-53.5)/6.5)]$$
$$\text{Win proxy} = \sigma(\text{margin} / \max(.5, 7\sqrt{\max(.02,\text{time_left/total})}))$$

Limits visible in code — Hand-set sigmoid constants are not calibrated probabilities. Missing kick distance returns 0.0. Some Scout text asserts tactical causes absent from the event record; Stat can substitute 2/3 points when score_delta is zero. Determinism does not guarantee truth. Test missed shots, overtime and missing clock/distance; return unknown instead of fabricated certainty.

4 · Evidence selection and recap generation



Truncating the union can drop a late decisive event; reserve its slot first.

_key_plays — Category weights include touchdown=10, dunk=9, three-pointer/interception=8; "miss" reduces weight. It unions highest-weight and late plays, deduplicates by Python object identity, sorts by ID and truncates. The final truncation can remove late plays it intended to preserve. ID ordering is an assumption about chronology; object identity is not a stable event ID.

Why select first — Selection compresses the prompt but can remove the decisive evidence. The writer receives _game_facts: teams/scores/date/status, period scores, lead changes, key plays and leaders. Improve with reserved late-event slots, explicit period/clock ordering and stable-ID deduplication; evaluate selection recall separately from prose quality.

Writer paths — General recap tries configured Anthropic models, then OpenAI, then _data_recap. generated_by labels llm vs data; response also exposes provider failure when available. Both scores are used to infer played status because the source status can be mislabeled. This is a heuristic, not proof a game has ended.

Grounding requirement — Every sentence needs a supported relation, not merely familiar names/numbers. If A has 30 points and B has 12, "B scored 30" is false even when all tokens occur in facts. A template reduces some freedom but can still misstate missing/tied/live data.

5 · Reels, two-host broadcasts and timing

Media architecture — Existing source video URLs supply footage; scene metadata, overlays and generated narration arrange it. Speech synthesis is text→audio; speech recognition is audio→text. Neither means footage was generated or tactical events were visually recognized. Missing clips should stay unavailable.

broadcast.py — Two hosts: play and analyst. Requested tiers: 120/300/600 seconds with 12/26/44 turns; budgets 2400/5200/8000 tokens. Turn schema: host, text, scene_type, duration_hint. Conditional trained writer → Anthropic fallback → deterministic broadcast. A turn count is a planning target, not measured spoken duration.

```
{host: "play", text: "...",
scene_type: "moment", duration_hint: 8}
```

Parsing and failure — _extract_turns parses the array, then salvages flat JSON objects if truncated. _clean_turns normalizes enums, cuts text at 400 chars, clamps hints to 4–20s; malformed int conversion can still fail. Regex salvage mishandles nested structures/braces in strings and cannot prove coherence or truth.

Better contract — Use schema-constrained output, evidence IDs, safe validation and explicit partial-result handling. The analyst prompt requests causal tactics that event descriptions may not support. Permit "not observed." Synchronize scenes to real audio completion; cancel stale speech before follow-up playback. Validate chronology, facts, media and timing independently.

6 · Personalization before writing

$$s = \sigma(b + w^T x);\ x=[\text{team, player, recency, magnitude, upcoming}]$$
$$\text{recency} = \exp(-\max(\text{age_days},0)/7);\ \text{half-life} = 7 \ln 2 \approx 4.85\ \text{d}$$

curation.py — Default bias −0.5; heuristic weights 2.2, 2.0, 1.3, 1.1, 0.6. Clip magnitude to [0,1]; team/player/upcoming are indicators. _weights loads curation_weights.json or heuristic-v1 and memoizes once per process. No exported weight file was found in the inspected tree. A version label does not establish learned provenance.

Ranking consequences — Sigmoid preserves raw-score order but does not calibrate click probability. At seven days recency≈.368, not .5. rank_items rounds to four decimals before sorting, which can create ties. Sort full precision, round for display. Newsletter then writes ranked evidence; a better writer cannot restore an omitted followed-team event.

Learned alternative — Collect exposure and interaction logs, handle cold-start fans explicitly, deduplicate game IDs, split by time/user as appropriate, compare NDCG to recency/popularity baselines. Synthetic labels from hand-chosen weights only recover those assumptions; they do not establish real-fan uplift.

7 · Distillation, LoRA and deployed evidence



Current training uses random records; grouped splitting is the improvement.

Offline pipeline — Facts → teacher drafts → grounding filter → chat-formatted pairs → adapter training → held-out evaluation → configured endpoint. Training files, saved weights, serving configuration and actual request logs establish different facts. Newsletter and broadcast have optional trained routes; do not claim every recap uses a fine-tuned model.

$$W' = W + (\alpha/r)BA;\ A{:}r\times d_{in},\ B{:}d_{out}\times r$$
$$\text{Trainable count } r(d_{in}+d_{out}),\ \text{versus } d_{in}\cdot d_{out} \text{ for full } W$$

Actual recipe — train_writer_lora.py: r=16, α=32, dropout=.05; q/k/v/o and gate/up/down projections; default lr=2e−4, 3 epochs, batch=1, accumulation=16, max sequence=4096, bf16, gradient checkpointing. For 4096² matrix: 131,072 adapter params vs 16,777,216 full (~0.78%). Checkpointing saves activations by recomputing; accumulation raises effective batch.

QLoRA and capacity — Optional NF4 4-bit frozen base, double quantization, bf16 compute; adapters still train. Saves weight/optimizer memory, not all activations or inference cost. Low rank limits update capacity; compare validation quality and throughput against LoRA/full tuning before choosing.

Leakage and filtering — Script uses random 95/5 records (seed 7), not game/time groups. grounding.check ignores numbers <13 and years, accepts either name token and allows one violation. It is a weak screen, not a guarantee. Split by game before generation; use a final time-disjoint test; check player–stat relationships and compare base LLM/template/adapter on identical facts.

Source map and recall — api/espn_public.py, play_analysis.py, recaps.py, reels.py, broadcast.py, curation.py, newsletter.py; db/store.py; training/grounding.py and train_writer_lora.py. Reconstruct the system as **facts → normalized records → selected evidence → labeled prose/media → validated state**. Blog sections 1–14 contain detailed examples and pinned excerpts.

8 · Identity, storage and the award race

Time	Request A	Request B
1 / 2	reads unresolved	reads unresolved
3 / 4	marks; awards +10	marks; awards +10
Atomic transition	check + mark + award	sees resolved; +0

Data model — Browser UUID in local storage is sent as X-Device-Id. Clearing storage or changing browser changes identity; a client-supplied ID is not authentication. Redis hashes store picks/profiles, sorted sets rank points, and saved newsletters use share tokens with expiry. Missing REDIS_URL makes storage operations no-ops, not durable success.

mark_resolved — Reads pick → checks unresolved → writes resolved → increments profile/streak and leaderboard separately. Two callers can both pass the check. A crash after marking resolved but before awarding can lose an award on retry. Individual Redis commands are atomic; the multi-command transition is not. The “exactly once” docstring overstates the implementation.

Correct alternative — Atomic Lua/transaction with conflict retry, or an idempotent award ledger plus reconciliation. Use device+game as logical operation identity and protect all counters in the same transition. Test concurrency and crashes at write boundaries. Distinguish unavailable store from already-resolved/no points.

9 · Evaluation: locate the error, then measure it

Stage	Measure / adversarial example
Adapter	Schema fixtures; missing score ≠ zero
Classification	Per-class precision/recall; missed shots
Selection	Key-event recall; decisive late play omitted
Writing	Supported claim rate; swapped player statistics
Broadcast	JSON + factuality + real playback timing
Curation	NDCG on fixed labeled candidates; cold start
State	Concurrent award, crash recovery, reload durability

$$\text{Precision} = TP/(TP+FP);\ \text{Recall} = TP/(TP+FN)$$
$$DCG = \Sigma(2^{\wedge}(\text{relevance})-1)/\log2(\text{rank}+1);\ NDCG = DCG/\text{ideal DCG}$$

Evidence boundary — No production factuality rate, trained-ranker uplift, cache hit rate or p95 is established by this review. Grounding-filter acceptance is not a quality metric: wrong relations and small fabricated numbers pass. Audit false accepts and false rejects on independently labeled claims.

10 · Serving decisions and alternatives

Bottlenecks — Source fetch, blocking model calls, cold cache and shared store can dominate. The async generation endpoint calls a synchronous recap function; blocking work can stall its event loop. _last_llm_error is module-global and can mix concurrent diagnostics. Use request-local error values and a suitable thread/job path.

Scale deliberately — Bound concurrency/retries; jitter retries; single-flight popular keys; separate live/final freshness; track source→selection→writer→store timings. GPUs are not the first fix for redundant HTTP or generation calls.